



BUBT

BANGLADESH UNIVERSITY OF
BUSINESS AND TECHNOLOGY

Commitment to Academic Excellence

LAB REPORT

Experiment Date : 13/09/2026

Experiment No : 01

Course Title : Operating System Lab

Course Code : CSE-210

Experiment Name : Lab Task

Submitted By:

Name : Md. Adnan Sami

ID No : 20255103068

Intake : 55

Section : 02

Program : B.Sc. in CSE

Submitted To:

Name : Mahbuba Habib

(Lecturer)

Dept. of CSE

**Bangladesh University of Business
& Technology**

Date of Submission : 17/09/2026

Problem 1: FCFS Scheduling

Problem Statement: Implement FCFS and find out its Completion Time, Turnaround Time & Waiting Time.

Process	Arrival Time	Burst Time
P1	1	5
P2	3	7
P3	2	9
P4	4	3
P5	2	3

Steps to Solve the Problem:

- ❖ Step 1: Take the number of processes and their Arrival Time and Burst Time as input.
- ❖ Step 2: Arrange the processes according to their Arrival Time.
- ❖ Step 3: Execute the processes using the FCFS scheduling algorithm.
- ❖ Step 4: Calculate the Completion Time (CT).
- ❖ Step 5: Calculate Turnaround Time using:
 $TAT = CT - AT$
- ❖ Step 6: Calculate Waiting Time using:
 $WT = TAT - BT$
- ❖ Step 7: Display Process ID, Arrival Time, Burst Time, Completion Time, Turnaround Time and Waiting Time.

Source Code:

```
1.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6
7      cout << "Enter number of processes: ";
8      cin >> n;
9
10     int pid[n], at[n], bt[n];
11     int ct[n], tat[n], wt[n];
12
13     cout << "Enter Arrival Time and Burst Time:\n";
14
15     for (int i = 0; i < n; i++) {
16         pid[i] = i + 1;
17         cout << "P" << pid[i] << ": ";
18         cin >> at[i] >> bt[i];
19     }
20
21     for (int i = 0; i < n - 1; i++) {
22         for (int j = 0; j < n - i - 1; j++) {
23             if (at[j] > at[j + 1]) {
24                 swap(at[j], at[j + 1]);
25                 swap(bt[j], bt[j + 1]);
26                 swap(pid[j], pid[j + 1]);
27             }
28         }
29     }
30
31     int time = 0;
32
33     for (int i = 0; i < n; i++) {
34         if (time < at[i])
35             time = at[i];
36
37         time = time + bt[i];
38
39         ct[i] = time;
40         tat[i] = ct[i] - at[i];
41         wt[i] = tat[i] - bt[i];
42     }
43
44     cout << "\nProcess\tAT\tBT\tCT\tTAT\tWT\n";
45
46     for (int i = 0; i < n; i++) {
47         cout << "P" << pid[i] << "\t"
48             << at[i] << "\t"
49             << bt[i] << "\t"
50             << ct[i] << "\t"
51             << tat[i] << "\t"
52             << wt[i] << endl;
53     }
54
55     return 0;
56 }
```

Output:

```
PS E:\CSE-209> g++ 1.cpp -o run
PS E:\CSE-209> ./run
Enter number of processes: 5
Enter Arrival Time and Burst Time:
P1: 1 8
P2: 3 7
P3: 2 9
P4: 4 3
P5: 2 3

Process AT    BT    CT    TAT   WT
P1      1      8      9      8     0
P3      2      9     18     16     7
P5      2      3     21     19    16
P2      3      7     28     25    18
P4      4      3     31     27    24
```

Problem 2: SJF Scheduling

Problem Statement: Implement SJF and find out its Completion Time, Turnaround Time & Waiting Time.

Process	Arrival Time	Burst Time
P1	1	5
P2	3	7
P3	2	9
P4	4	3
P5	2	3

Steps to Solve the Problem:

- ❖ Step 1: Take the number of processes and their Arrival Time and Burst Time as input.
- ❖ Step 2: Check which processes have arrived at the current time.
- ❖ Step 3: Among the available processes, select the process with the shortest Burst Time.
- ❖ Step 4: Execute the selected process completely.
- ❖ Step 5: Calculate the Completion Time (CT).
- ❖ Step 6: Calculate Turnaround Time using:
 $TAT = CT - AT$
- ❖ Step 7: Calculate Waiting Time using:
 $WT = TAT - BT$
- ❖ Step 8: Repeat the process until all processes are completed.
- ❖ Step 9: Display the final result.

Source Code:

```
2.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6
7      cout << "Enter number of processes: ";
8      cin >> n;
9
10     int pid[n], at[n], bt[n];
11     int ct[n], tat[n], wt[n];
12     int completed[n] = {0};
13
14     cout << "Enter Arrival Time and Burst Time:\n";
15
16     for (int i = 0; i < n; i++) {
17         pid[i] = i + 1;
18         cout << "P" << pid[i] << ": ";
19         cin >> at[i] >> bt[i];
20     }
21
22     int time = 0;
23     int count = 0;
24
25     while (count < n) {
26
27         int index = -1;
28
29         for (int i = 0; i < n; i++) {
30             if (completed[i] == 0 && at[i] <= time) {
31
32                 if (index == -1 || bt[i] < bt[index]) {
33                     index = i;
34                 }
35             }
36         }
37
38         if (index == -1) {
39             time++;
40         }
41         else {
42             time = time + bt[index];
43
44             ct[index] = time;
45             tat[index] = ct[index] - at[index];
46             wt[index] = tat[index] - bt[index];
47
48             completed[index] = 1;
49             count++;
50         }
51     }
52
53     cout << "\nProcess\tAT\tBT\tCT\tTAT\tWT\n";
54
55     for (int i = 0; i < n; i++) {
56         cout << "P" << pid[i] << "\t"
57             << at[i] << "\t"
58             << bt[i] << "\t"
59             << ct[i] << "\t"
60             << tat[i] << "\t"
61             << wt[i] << endl;
62     }
63
64     return 0;
65 }
```

Output:

```
PS E:\CSE-209> g++ 2.cpp -orun
PS E:\CSE-209> ./run
Enter number of processes: 5
Enter Arrival Time and Burst Time:
P1: 1 5
P2: 3 7
P3: 2 9
P4: 4 3
P5: 2 3
```

Process	AT	BT	CT	TAT	WT
P1	1	5	6	5	0
P2	3	7	19	16	9
P3	2	9	28	26	17
P4	4	3	9	5	2
P5	2	3	12	10	7

Please Turn Over →